

Contents

Supplementary Material S5: Tier 3 CI/CD Demonstration	1
GitHub Actions workflow	1
Consistency-check script	2
What the check catches	3
Extending the workflow	3

Supplementary Material S5: Tier 3 CI/CD Demonstration

This supplementary illustrates a minimal Tier 3 (Advanced) workflow as described in Section 3 of the manuscript. It implements three of the capabilities listed under Tier 3:

1. **Continuous rendering of PlantUML diagrams** on every push to the repository, so that figures in documentation never drift from the source .puml files.
2. **Automated consistency checking** — lexical detection of references in state-machine and activity-diagram guards to attributes that are not declared in the class diagram. (This is the kind of inconsistency that motivated the addition of the moisture attribute to the reference framework during peer review.)
3. **Versioned diagram artefacts** — rendered SVG/PDF deposited as build artefacts attached to each commit, so that historical figure versions can be retrieved without re-running the rendering pipeline.

The workflow is intentionally small and is intended as a starting point that practitioners can extend. It is *not* a substitute for full UML model verification.

GitHub Actions workflow

Place the following at .github/workflows/render.yml:

```
name: Render and verify UML diagrams

on:
  push:
    paths:
      - 'plantuml/**/*.puml'
      - 'scripts/**'
      - '.github/workflows/render.yml'
  pull_request:
    paths:
      - 'plantuml/**/*.puml'

jobs:
  render:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install PlantUML and Inkscape
        run: |
          sudo apt-get update
          sudo apt-get install -y plantuml inkscape
```

- name: Render diagrams (SVG + PDF)
run: bash scripts/render_figures.sh
- name: Consistency check – guard attributes are declared
run: bash scripts/check_consistency.sh
- name: Upload rendered diagrams
uses: actions/upload-artifact@v4
with:
 name: uml-figures
 path: plantuml/figures/

Each push that touches a .puml file or the rendering scripts triggers the workflow. The job installs PlantUML and Inkscape, regenerates all diagrams, runs the consistency check, and uploads the resulting figures as a downloadable artefact. A failed consistency check fails the build and blocks merging until the discrepancy is resolved.

Consistency-check script

scripts/check_consistency.sh extracts attribute identifiers from the class diagram and verifies that every guard expression in the state-machine and activity diagrams references only declared attributes:

```
#!/usr/bin/env bash
set -euo pipefail

CLASS_DIAGRAM="plantuml/AedesClassDiagram.puml"
GUARDED=(
  "plantuml/AedesStateMachine.puml"
  "plantuml/AedesActivityDiagram.puml"
)

# Reserved tokens that legitimately appear in guards but are not class
# attributes (Boolean operators, environmental drivers passed as
# parameters, common loop counters).
RESERVED='^(AND|OR|NOT|true|false|temp|photoperiod|degreedays|degreeDays|age|density|threshold|

# Extract attribute names from class diagram
# (lines like " - attrName: type" or " + methodName(...)"
attrs=$(grep -E '^\s*[-+]\s+\w+' "$CLASS_DIAGRAM" \
| sed -E 's/^\s*[-+]\s+([A-Za-z_][A-Za-z0-9_]*)\.*/\1/' \
| sort -u)

status=0
for f in "${GUARDED[@]}; do
  # Identifiers used inside guard brackets [...]
  used=$(grep -oE '\[[^\]]+\]' "$f" \
| grep -oE '[A-Za-z_][A-Za-z0-9_]*' \
| sort -u)
  for id in $used; do
    [[ "$id" =~ $RESERVED ]] && continue
    if ! grep -qx "$id" <<< "$attrs"; then
      echo "WARN: $f references '$id' not declared in $CLASS_DIAGRAM" >&2
    fi
  done
done
```

```

        status=1
    fi
done
done

if [[ $status -eq 0 ]]; then
    echo "Consistency check passed."
fi
exit $status

```

Made executable with `chmod +x scripts/check_consistency.sh`.

What the check catches

The check is a **lexical heuristic**, not a semantic UML validation. It catches the following classes of error:

- Attributes referenced in a guard but never declared in the class diagram (e.g., moisture referenced before the class diagram is updated).
- Typos that introduce a new identifier silently (e.g., `degreDays` instead of `degreeDays`).
- Renames in one diagram not propagated to others.

It does **not** catch:

- Type errors (e.g., comparing a string to a float).
- Multiplicity violations (e.g., a single `Egg` instance assigned to multiple `BreedingSite` instances when the diagram says "many" `--o` "1").
- Dynamic-semantic errors (e.g., a state machine with unreachable states or non-deterministic transitions).

For higher-assurance settings, the workflow should be extended with a proper UML model parser such as `pyuml2` or the Eclipse UML2 toolkit, together with a unit-test suite that instantiates the generated skeleton code against synthetic inputs.

Extending the workflow

Practitioners adopting this Tier 3 setup may wish to add:

Step	Purpose
pandoc rendering of supplementary .md to PDF	Keep .pdf artefacts synchronised with editable .md sources.
pytest over generated Python skeletons	Verify that the skeleton at least imports and instantiates.
Rscript -e "rcmdcheck::rcmdcheck()" over generated R packages	Validate package structure for the matrix model skeleton.
Diagram diff on pull requests (e.g., <code>plantuml-diff</code>)	Make structural changes reviewable as visual diffs.
Versioned releases via softwareheritage or zenodo-upload	Provide citable archival snapshots.

Document version: 1.0 Last updated: 2026